

RAG Pipeline - Agentic RAG: Corrective RAG (CRAG)

1. Definition

Corrective RAG (CRAG) is an "Active" or "Agentic" RAG strategy that introduces a **self-correction mechanism**.

Instead of blindly trusting the documents retrieved from the vector store, CRAG uses a lightweight "Evaluator" (an LLM) to grade the quality of the retrieved documents. Based on this grade, it dynamically decides whether to use the documents, refine them, or discard them and search the web instead.

2. Core Philosophy: The "State Machine"

CRAG moves beyond simple chains. It operates as a **State Machine** (implemented via **LangGraph**) where the LLM decides the transitions between steps.

- **Retrieval:** Fetch documents.
- **Evaluation:** Judge relevance.
- **Decision:** Route to "Generate," "Refine," or "Web Search."

3. The Architecture

A. The Evaluator (Grading)

Every retrieved document is passed to an LLM grader (e.g., a small, fast model) that outputs a confidence score or label:

1. **Correct:** The document answers the query.
2. **Ambiguous:** The document might be relevant but is unclear or incomplete.
3. **Incorrect:** The document is irrelevant (noise).

B. Path 1: Correct Documents (Knowledge Refinement)

If a document is "Correct," CRAG doesn't just pass it to the generator. It performs **Knowledge Refinement**:

1. **Decomposition:** The document is broken down into fine-grained "**Knowledge Strips**" (e.g., individual sentences or 1-2 sentence units).
2. **Filtering:** The evaluator grades *each strip* individually.
3. **Recomposition:** Only the relevant strips are concatenated.
 - *Benefit:* This removes irrelevant noise *within* a relevant document, ensuring the final context is highly precise.

C. Path 2: Incorrect/Ambiguous Documents (Web Search)

If the documents are "Incorrect" (all noise) or "Ambiguous" (unsure):

1. **Fallback:** The system triggers a **Web Search** tool (e.g., Tavily, Google Search).
2. **Query Rewriting:** The system often rewrites the user's query to be more suitable for a search engine before executing the search.
3. **Supplement:** The web results are used to replace or supplement the vector store data.

4. Key LangChain & LangGraph Tools

Component	Tool / Implementation
State Machine	LangGraph : The core framework. You define a <code>StateGraph</code> with nodes for <code>retrieve</code> , <code>grade</code> , and <code>generate</code> . You use <code>conditional_edges</code> to route between them based on the grade.
The Evaluator	<code>ChatPromptTemplate</code> + <code>PydanticOutputParser</code> : You create a prompt asking the LLM to output a binary score (<code>yes / no</code>) or a specific grade, enforced by Pydantic.
Web Search	<code>TavilySearchResults</code> : The standard tool in LangChain for LLM-optimized web search. It returns clean text content rather than raw HTML.
Knowledge Strips	<code>RecursiveCharacterTextSplitter</code> : Used again <i>post-retrieval</i> to split the retrieved doc into tiny segments (strips) for the refinement phase.

5. When to use CRAG?

- **High-Stakes Accuracy:** When "hallucinations" or irrelevant context are unacceptable.
- **Dynamic Data:** When your internal knowledge base might be incomplete or out-of-date, and you want the system to automatically "fall back" to the web.
- **Complex Queries:** When documents contain a mix of relevant and irrelevant info, and you need to strip out the noise.